МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Державний університет "Київський авіаційний інститут"
Факультет комп’ютерних наук та технологій
Кафедра комп’ютерних інформаційних технологій

Звіт з лабораторної роботи 2
з дисципліни: Криптологія
“Генерація псевдовипадкових послідовностей”

Виконав:
Студент групи Б-113-23-1-ПМ
Владєєв Іван Глібович
Прийняла:
Кандидат технічних наук, доцент
кафедри ПМ
Чолишкіна Ольга Геннадіївна

Київ - 2026

Мета роботи – ознайомитись з методами генерації псевдовипадкових послідовностей, джерелами ентропії та тестами на випадковість. Навчитись реалізовувати генератори псевдовипадкових послідовностей з джерелом ентропії для генерації криптографічних ключів.

Зв’язок з результатами навчання:

Використовувати генератори псевдовипадкових послідовностей для генерації криптографічних ключів, використовувати генератори гами для алгоритмів потокового шифрування..

Постановка задачі.

1. Ознайомитись з методами генерації псевдовипадкових послідовностей Лемера та BBS, сучасними тестами на випадковість (NIST).

2. Реалізувати програмно алгоритм Лемера, BBS. Зробити перевірку тестування (тести NIST).

3. Додати власне джерело ентропії у код генератора та перевірити вихід за допомогою тестів.

4. Порівняти виходи генератору BBS та генератора з ентропією. Зробити висновки та надати рекомендації.

5. Оформити звіт до лабораторної роботи, включивши в нього теоретичні відомості, скріни програмного забезпечення, аналіз результатів та висновки.

Хід роботи

1. Реалізувати програмно алгоритм Лемера, BBS, перевірити тестами.

Алгоритм Лемера - це класичний лінійний конгруентний генератор псевдовипадкових чисел (Linear Congruential Generator) з інкрементом c = 0.

Алгоритм можна зручно виразити у вигляді наступного рекурентного відношення:

$$X_{k+1} = a \cdot X_k \bmod m$$

X - це послідовність псевдовипадкових чисел.

a - це множник

mod m - це та частина, що переносить нас в модульну арифметику, де ми постійно рухаємось навколо певного числа m, називаємого модулем.

А також зерно (“seed”) - X_0, що є початковим значенням генератора.

Правила вибору параметрів:

m - велике просте число або степінь 2. Зазвичай $2^{31} - 1$ (2147483647).

a (множник) - має бути "примітивним коренем" по модулю m. Тобто послідовність a^1, a^2, ... повинна обходити всі значення від 1 до m−1. Для m = $2^{31}-1$. Перевірені варіанти - 16807 або 48271

X_0 - може бути будь-яке від 1 до m−1.

Програмна реалізація алгоритму має вигляд:

```
def LemerGenerator(self, n, multi, modulus, seed):
    assert 0 < seed < modulus, "seed must be in range (0, modulus)"
    prev = seed
    generated_sequence = []
    for i in range(n):
        value = (multi * prev) % modulus
        generated_sequence.append(value)
        prev = value
    return generated_sequence
```

Рис. 1

Для прикладу згенеруємо 10 чисел.

lemer_generated = num_gen.LemerGenerator(n=10, multi=16807, modulus=2147483647, seed=10)

print(f"generated sequence: {lemer_generated}"):

generated sequence: [168070, 677268843, 1194115201, 1259501992, 703671065, 407145426, 1010275440, 1693606898, 1702877348, 745024267]

Nist потребує строкове представлення числа у вигляді байтів, для переведення з числового в бітове була написана відповідна функція:

```
@staticmethod
def numbers_to_bits(numbers, bit_width=31):
    bits = []
    for number in numbers:
        bits.append(format(number, f'0{bit_width}b'))
    return ''.join(bits)
```

Рис. 2

Тепер протестуємо отриману послідовність за допомогою NIST:

```
RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES
------------------------------------------------------------------------------
   generator is </home/siercig/University/Cryptography/LabTwo/lemer_bits.txt>
------------------------------------------------------------------------------
 C1  C2  C3  C4  C5  C6  C7  C8  C9 C10  P-VALUE  PROPORTION  STATISTICAL TEST
------------------------------------------------------------------------------
  4   4   8   6   7   3   7   8   4   4  0.637119     54/55      Frequency
  5   1   9   4   9   7   5   7   5   3  0.202268     54/55      BlockFrequency
  4   4   6   5   1   6  12   7   3   7  0.062821     54/55      CumulativeSums
  3   5   7   6   8   4   3   7   6   6  0.759756     53/55      CumulativeSums
  1   3   5   9   6   7   4   8   6   6  0.304126     55/55      Runs
  6   3   1   8   9   5   6   5   5   7  0.334538     55/55      LongestRun
  5   3   3   4   6  10   3   8   4   9  0.162606     55/55      Rank
  8   4   8   6   6   4   9   2   2   6  0.249284     54/55      FFT
  3   2   4   8   5   7   7   8   6   5  0.514124     55/55      NonOverlappingTemplate
  5   9   4   5   6   4   6   5   7   4  0.834308     53/55      NonOverlappingTemplate
  2   5   9   7   4   2   7   6   9   4  0.202268     55/55      NonOverlappingTemplate
  5   4   4   8   5   3  11   4   7   4  0.249284     55/55      NonOverlappingTemplate
  5   5   2   4   9   6   3   8   8   5  0.366918     54/55      NonOverlappingTemplate
  5   7   3   8   2   7   1   6   4  12  0.021999     54/55      NonOverlappingTemplate
  4   5   6   4   7  12   6   3   4   4  0.181557     55/55      NonOverlappingTemplate
  5   2   8   5   8   4   8   6   4   5  0.554420     55/55      NonOverlappingTemplate
  8   2   4   7   4   5   8   5   9   3  0.304126     55/55      NonOverlappingTemplate
  7   5   5   4   5   4   5   5   7   8  0.924076     55/55      NonOverlappingTemplate
  5   6   7   4   6   4   9   3   5   6  0.759756     55/55      NonOverlappingTemplate
  6   7   5   4   2   6   5   4   7   9  0.595549     55/55      NonOverlappingTemplate
  5   8   4   3   5   5   6   7   8   4  0.759756     55/55      NonOverlappingTemplate
  7   6   2   5  10   6   4   4   6   5  0.474986     55/55      NonOverlappingTemplate
  8   6   5   7   7   3   3   4   6   6  0.759756     54/55      NonOverlappingTemplate
  5   4   8   7   2   9   7   3   3   7  0.275709     55/55      NonOverlappingTemplate
  7   9   4   4   3   2   6   5   8   7  0.366918     55/55      NonOverlappingTemplate
  2   8   4   5   3   4   8   8   4   9  0.224821     55/55      NonOverlappingTemplate
  4   7   6   4   6   4   8   4   3   9  0.554420     55/55      NonOverlappingTemplate
  4   7   3   7   7   2   7   5   5   8  0.554420     54/55      NonOverlappingTemplate
  5   6   4   6   9   7   5   3   5   5  0.798139     53/55      NonOverlappingTemplate
  3  10   3   5   8   9   7   5   3   2  0.090936     54/55      NonOverlappingTemplate
  4   3   6   6  11   5   8   2   4   6  0.181557     55/55      NonOverlappingTemplate
  4   5   5   5   7   5   5   4  12   3  0.224821     55/55      NonOverlappingTemplate
  6   6   6   5   2   9   4   6   4   7  0.637119     53/55      NonOverlappingTemplate
  1   6   5   8   2   6  12   4   5   6  0.042808     54/55      NonOverlappingTemplate
```

Рис.3

6	11	5	3	5	4	7	4	7	3	0.275709	53/55	NonOverlappingTemplate
4	3	4	9	8	2	4	5	10	6	0.145326	55/55	NonOverlappingTemplate
7	4	8	7	5	9	3	2	3	7	0.275709	52/55	NonOverlappingTemplate
5	9	4	8	8	2	7	5	3	4	0.304126	55/55	NonOverlappingTemplate
8	6	3	6	4	4	9	5	4	6	0.637119	55/55	NonOverlappingTemplate
7	5	6	6	9	5	6	2	9	0	0.102526	55/55	NonOverlappingTemplate
6	7	1	7	7	1	10	8	6	2	0.037566	55/55	NonOverlappingTemplate
7	3	7	4	4	4	5	8	4	9	0.514124	54/55	NonOverlappingTemplate
8	5	6	4	4	5	7	5	5	6	0.946308	54/55	OverlappingTemplate
4	5	6	9	3	4	7	7	5	5	0.719747	54/55	Universal
4	8	7	3	2	14	4	3	5	5	0.007160	54/55	ApproximateEntropy
4	2	2	6	6	4	2	5	2	1	0.299251	32/34	RandomExcursions
3	5	3	5	2	4	4	5	1	2	0.671779	33/34	RandomExcursions
1	5	3	3	4	4	5	1	5	3	0.602458	34/34	RandomExcursions
3	2	4	3	2	4	4	6	4	2	0.804337	34/34	RandomExcursions
1	4	2	4	4	4	4	2	5	4	0.804337	33/34	RandomExcursions
3	2	0	1	4	5	9	2	6	2	0.008879	33/34	RandomExcursions
3	1	6	3	3	5	3	5	4	1	0.468595	34/34	RandomExcursions
3	6	2	2	5	6	2	1	2	5	0.253551	34/34	RandomExcursions
6	3	2	7	2	1	6	0	3	4	0.054199	34/34	RandomExcursionsVariant
7	2	2	5	1	5	4	3	3	2	0.299251	34/34	RandomExcursionsVariant
8	3	1	5	3	5	4	1	2	2	0.100508	34/34	RandomExcursionsVariant
8	3	3	3	3	5	3	3	1	2	0.253551	34/34	RandomExcursionsVariant
8	5	1	3	3	1	3	2	4	4	0.148094	33/34	RandomExcursionsVariant
6	4	1	6	4	3	1	2	4	3	0.350485	33/34	RandomExcursionsVariant
5	4	5	1	4	4	2	2	4	3	0.739918	33/34	RandomExcursionsVariant
4	5	6	2	3	2	2	3	3	4	0.739918	34/34	RandomExcursionsVariant
2	2	6	5	3	7	0	4	2	3	0.122325	34/34	RandomExcursionsVariant
2	4	5	4	2	4	3	5	2	3	0.862344	34/34	RandomExcursionsVariant
3	2	3	2	7	2	6	5	2	2	0.253551	33/34	RandomExcursionsVariant
1	2	4	2	5	4	4	3	6	3	0.602458	33/34	RandomExcursionsVariant
0	4	3	2	2	4	2	5	7	5	0.178278	34/34	RandomExcursionsVariant
0	5	4	2	3	2	4	2	4	8	0.100508	34/34	RandomExcursionsVariant
1	5	4	3	2	3	4	4	3	5	0.804337	34/34	RandomExcursionsVariant
2	3	2	5	5	3	5	2	3	4	0.804337	34/34	RandomExcursionsVariant
2	2	4	5	4	8	1	0	4	4	0.066882	34/34	RandomExcursionsVariant
4	1	3	6	7	3	3	2	2	3	0.299251	33/34	RandomExcursionsVariant
5	7	6	9	5	3	2	5	6	7	0.554420	55/55	Serial
6	5	6	9	8	6	5	3	0	7	0.202268	54/55	Serial
3	9	7	7	2	8	4	4	3	8	0.202268	54/55	LinearComplexity

Рис. 4

Статистичні показники Генератор (константи MINSTD) успішно пройшов усі перевірки. Пропорція проходження тестів у нормі (переважно 54/55 або 55/55), P-значення розподілені рівномірно. Для звичайного програмного моделювання такої псевдовипадковості цілком достатньо.

Причина успіху - результат використання роками оптимізованих констант Парка-Міллера. modulus = 2147483647, multi = 16807. NIST STS шукає лише базові статистичні аномалії (наприклад, перекіс між нулями та одиницями), з чим алгоритм чудово справляється завдяки взяттю остачі по великому простому модулю.

Криптографічна вразливість генератора Лемера зумовлена його абсолютно лінійною математичною моделлю, яка описується рівнянням $X_{n+1}=(a \cdot X_n)(\mathrm{mod}\, m)$.

Оскільки константи множника a та модуля m є публічно відомими (у даному випадку це стандартизовані значення MINSTD), алгоритм не має прихованого внутрішнього стану.

Якщо зловмисник перехоплює хоча б одне згенероване число X_n, він отримує єдину невідому змінну в цьому рівнянні.

Це дозволяє йому миттєво обчислити всі наступні значення алгоритму шляхом звичайного множення, а всі попередні значення - через обчислення оберненого елемента за модулем. Таким чином, компрометація лише одного числа призводить до повного розкриття всієї псевдовипадкової послідовності в обидва боки без необхідності складного криптоаналізу чи повного перебору.

Алгоритм BBS (Blum Blum Shub)

BBS - криптографічно стійкий генератор псевдовипадкових чисел, безпека якого тримається на складності факторизації великих чисел

Для реалізації беруться два великих простих числа p і q з умовою $p \equiv 3 \pmod 4$. Їх добуток $n = p \times q$ використовується як модуль у всіх операціях.

Початкове значення x_0 має бути взаємно простим з n, тобто НСД(x_0, n) = 1. Тримається в секреті — весь генератор залежить від нього.

Кожен наступний стан: $x_{i+1} = x_i^2 \bmod n$

З кожного x_i беремо молодший біт це і буде вихідний біт послідовності. Наприклад, x = 110110 → біт 0.

```python
def BBS(self, p, q, number_of_bits):
    if p % 4 != 3 or q % 4 != 3 or p == q:
        print("Повинні виконуватись умови ...")
        return
    n = p*q

    if math.gcd(seed, n) != 1:
        print("Числа повинні бути взаємно простими (не мати спільних дільників окрім 1) ...")
        return
    x = pow(seed, 2, n)
    bits = []
    for _ in range(number_of_bits):
        x = pow(x, 2, n)
        bits.append(str(x & 1))
    return "".join(bits)
```

Рис. 2. Програмна реалізація BBS.

Таким чином ми генеруємо псевдовипадкові біти:

BBS bits: 1000000

Перші 64:
1101001000101001101000110001100101111011000110010100000101011111

Проте ми хотіли не біти, а числа, щоб конвертувати, було написано функцію, що бере bitstring і ріже його на числа фіксованої довжини.

Наприклад: 32 біти - це одне число, обрізаємо по 32, або можемо взяти 8, обрізати по 8 і т.д.

```python
@staticmethod
def bitsream_to_numbers(bitstream, chunk_size):
    numbers = []
    for _ in range(0, len(bitstream), chunk_size):
        binary_string = bitstream[_:_+chunk_size]
        number = int(binary_string, 2)
        numbers.append(number)
    return numbers
```

Рис. 3.

BBS bits: 1000000

Перші 64:

1100100111000010101101111111111101101100010011010000101110100101

Кількість чисел: 31250

Перші 10 чисел: [3384981503, 1816988581, 2193754220, 3054775364, 3752717172, 154189410, 2394814702, 3521708004, 63894810, 2333219937]

Протестуємо послідовність через Nist, як ми це робили у випадку з Лемером:

RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES

generator is </home/siercig/University/Cryptography/LabTwo/bbs_bits.txt>

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION		STATISTICAL TEST
5	5	7	5	3	3	8	13	3	3	0.028817	55/55		Frequency
5	9	7	3	5	5	4	4	7	6	0.719747	55/55		BlockFrequency
9	2	5	4	6	6	5	3	6	9	0.366918	55/55		CumulativeSums
5	3	5	6	6	7	3	4	6	10	0.514124	55/55		CumulativeSums
2	6	2	6	8	7	6	4	3	11	0.090936	55/55		Runs
8	3	3	7	4	9	7	3	7	4	0.334538	55/55		LongestRun
6	1	8	3	4	6	8	9	4	6	0.224821	53/55		Rank
5	5	6	6	7	2	5	10	4	5	0.514124	55/55		FFT
6	8	4	8	4	4	7	5	6	3	0.719747	55/55		NonOverlappingTemplate
7	8	4	9	3	1	5	5	6	7	0.275709	55/55		NonOverlappingTemplate
11	7	3	4	4	5	7	5	5	4	0.334538	53/55		NonOverlappingTemplate
5	4	4	5	10	8	8	1	7	3	0.129620	54/55		NonOverlappingTemplate
3	8	4	3	5	12	7	4	7	2	0.048716	54/55		NonOverlappingTemplate
4	4	8	5	4	7	4	6	8	5	0.798139	55/55		NonOverlappingTemplate
7	7	7	2	4	11	5	6	5	1	0.090936	54/55		NonOverlappingTemplate
7	4	4	6	6	2	7	5	6	8	0.719747	53/55		NonOverlappingTemplate
7	4	8	7	8	2	6	5	5	3	0.514124	54/55		NonOverlappingTemplate
5	2	5	10	4	5	8	5	6	5	0.437274	55/55		NonOverlappingTemplate
7	6	9	3	6	3	4	8	2	7	0.304126	55/55		NonOverlappingTemplate
4	10	4	8	3	3	7	7	3	6	0.249284	55/55		NonOverlappingTemplate
4	6	4	6	7	2	10	3	5	8	0.275709	54/55		NonOverlappingTemplate
4	7	6	6	4	4	8	7	4	5	0.867692	54/55		NonOverlappingTemplate
7	5	5	3	8	5	9	1	4	8	0.224821	55/55		NonOverlappingTemplate
3	3	11	6	5	6	7	6	7	1	0.115387	54/55		NonOverlappingTemplate
9	8	8	3	1	4	12	2	2	6	0.003447	55/55		NonOverlappingTemplate
6	9	6	5	7	5	1	6	5	5	0.554420	53/55		NonOverlappingTemplate
3	6	6	7	6	1	9	5	4	8	0.304126	54/55		NonOverlappingTemplate
13	1	6	5	3	7	5	4	3	8	0.014550	51/55	*	NonOverlappingTemplate
2	7	8	6	8	4	8	5	2	5	0.334538	55/55		NonOverlappingTemplate
5	6	6	4	6	6	7	8	3	4	0.867692	53/55		NonOverlappingTemplate
3	4	9	7	6	2	7	10	4	3	0.129620	54/55		NonOverlappingTemplate
8	4	5	6	5	2	3	9	5	8	0.366918	54/55		NonOverlappingTemplate
8	3	8	5	7	9	3	3	4	5	0.334538	55/55		NonOverlappingTemplate

Рис. 4

2.1.2 > sts-2.1.2 > experiments > AlgorithmTesting > bbs.txt

5	5	2	7	3	6	5	11	4	7	0.224821	55/55	NonOverlappingTemplate
9	4	6	6	4	5	9	5	2	5	0.437274	54/55	NonOverlappingTemplate
2	9	5	10	2	7	4	4	9	3	0.048716	55/55	NonOverlappingTemplate
2	5	6	8	4	6	5	5	4	10	0.401199	55/55	NonOverlappingTemplate
9	11	0	4	5	5	6	6	7	2	0.028817	53/55	NonOverlappingTemplate
6	4	6	7	7	4	6	4	6	5	0.964295	54/55	NonOverlappingTemplate
6	2	6	9	7	2	8	7	5	3	0.249284	55/55	OverlappingTemplate
7	6	3	4	5	3	8	10	2	7	0.202268	55/55	Universal
4	3	11	8	7	2	3	7	6	4	0.102526	55/55	ApproximateEntropy
1	2	2	7	6	1	4	4	1	1	0.001691	29/29	RandomExcursions
0	1	6	6	3	2	1	5	2	3	0.003577	29/29	RandomExcursions
2	1	5	4	3	2	4	3	4	1	0.186566	29/29	RandomExcursions
1	2	7	0	2	4	4	1	3	5	0.003577	29/29	RandomExcursions
0	2	6	3	3	4	2	2	2	5	0.041438	29/29	RandomExcursions
0	4	3	3	2	4	2	4	4	3	0.242986	29/29	RandomExcursions
4	2	3	2	2	4	2	5	2	3	0.392456	28/29	RandomExcursions
1	2	2	4	4	6	2	1	3	4	0.078086	29/29	RandomExcursions
1	4	3	5	3	3	3	1	4	2	0.242986	29/29	RandomExcursionsVariant
2	3	2	6	3	2	3	4	1	3	0.186566	29/29	RandomExcursionsVariant
4	3	1	4	1	3	6	0	4	3	0.029796	29/29	RandomExcursionsVariant
4	3	2	4	3	2	1	1	4	5	0.186566	29/29	RandomExcursionsVariant
4	4	2	3	1	1	2	3	5	4	0.186566	28/29	RandomExcursionsVariant
5	1	5	0	1	2	4	5	3	3	0.021262	29/29	RandomExcursionsVariant
2	4	2	2	4	0	6	2	2	5	0.029796	29/29	RandomExcursionsVariant
2	1	2	3	3	4	2	2	4	6	0.141256	29/29	RandomExcursionsVariant
1	4	3	0	6	0	5	6	1	3	0.000787	29/29	RandomExcursionsVariant
3	3	2	2	6	4	3	1	3	2	0.186566	29/29	RandomExcursionsVariant
3	3	0	4	2	5	4	3	3	2	0.186566	29/29	RandomExcursionsVariant
3	1	2	4	3	5	3	2	3	3	0.392456	29/29	RandomExcursionsVariant
3	2	1	4	4	0	3	5	3	4	0.105618	28/29	RandomExcursionsVariant
2	3	1	3	2	4	1	3	7	3	0.041438	28/29	RandomExcursionsVariant
4	0	4	3	0	2	4	3	6	3	0.021262	28/29	RandomExcursionsVariant
3	5	1	1	4	2	4	5	3	1	0.078086	29/29	RandomExcursionsVariant
4	3	3	2	3	4	4	4	1	1	0.311542	29/29	RandomExcursionsVariant
3	6	2	2	2	3	5	2	1	3	0.105618	29/29	RandomExcursionsVariant
9	6	5	6	4	3	5	5	3	9	0.474986	54/55	Serial
4	10	5	4	8	8	3	6	3	4	0.275709	55/55	Serial
4	4	6	9	7	2	7	6	5	5	0.595549	55/55	LinearComplexity

Рис. 5

Генератор BBS успішно пройшов серію тестів NIST STS. Розподіл P-значень є достатньо рівномірним, а відсоток проходження більшості тестів становить 54/55 або 55/55. Незначне відхилення (51/55) в одному з підтестів шаблонів є допустимою статистичною похибкою. Згенерований бітовий потік демонструє високі показники псевдовипадковості, не поступаючись оптимізованим лінійним алгоритмам.

Криптографічна стійкість Головна відмінність алгоритму BBS від генератора Лемера полягає у його доведеній криптографічній стійкості. Його безпека ґрунтується на обчислювальній складності задачі факторизації великих чисел. Формула переходу алгоритму $X_{n+1} = (X_n)^2 \bmod M$ використовує математичну незворотність операції

добування квадратного кореня за модулем складеного числа M, яке є добутком двох простих чисел p і q.

Навіть володіючи повною згенерованою послідовністю, зловмисник алгоритмічно не здатний передбачити наступні біти або відновити попередні стани без знання секретних дільників p та q. Алгоритм не має уразливостей до лінійного криптоаналізу. Єдиним вектором атаки є пряма факторизація модуля M. При використанні M = 1024 або 2048 біт злам стає обчислювально неможливим.

BBS забезпечує не лише статистичну рівномірність, але й абсолютну криптографічну надійність. На відміну від лінійних конгруентних генераторів, компрометація частини потоку BBS не призводить до розкриття всієї системи.

3. Генератор з власним джерелом ентропії

У цьому варіанті використовується генератор BBS, але початкове значення формується на основі зовнішньої ентропії та генератора Лемера. Спочатку генерується набір чисел за допомогою генератора Лемера за формулою $x(n+1) = (a \cdot x(n)) \bmod m$.
Отримані значення використовуються як додатковий компонент ентропії.

Далі для кожного потоку виконується отримання поточної ціни XMR/USDT з API Binance та зчитування поточного часу в наносекундах. Після цього обчислюється значення ентропії як побітове XOR трьох величин: ціни (без десяткової крапки), мітки часу та числа з Лемера. Таким чином формується “сире” значення, яке використовується для створення початкового seed.

```python
@staticmethod
def get_entropy_from_binance(lemer_value):
    url = "https://api.binance.com/api/v3/ticker/price?symbol=XMRUSDT"
    response = requests.get(url)
    price = response.json()['price']

    price_int = int(price.replace('.', ''))
    timestamp = time.time_ns()
    entropy = price_int ^ timestamp ^ lemer_value
    return entropy
```

Рис. 6

Отримане число приводиться до допустимого діапазону за формулою $s = (\text{entropy} \bmod (n - 2)) + 2$, де $n = p \cdot q$.

Далі запускається стандартний алгоритм BBS.

```python
raw_entropy = num_gen.get_entropy_from_binance(lemer_entropy_sequence[i])

s = (raw_entropy % (p * q - 2)) + 2

while math.gcd(s, p * q) != 1:
    s = (s + 1) % (p * q)

bits = num_gen.BBS(p=p, q=q, seed=s, number_of_bits=bits_per_stream)
if bits:
    file.write(bits + '\n')

print(f"BBS Binance Stream {i + 1}/{num_streams_bbs} | seed={s}")
time.sleep(0.1)
```

Рис. 7

На кожному кроці береться молодший біт числа x(i) (x mod 2), який додається до вихідної бітової послідовності

У підсумку BBS залишається основним криптографічним генератором, а його початкове значення формується з поєднання ринкових даних,

системного часу та псевдовипадкового числа Лемера, що підвищує непередбачуваність стартового стану.

generator is </home/siercig/University/Cryptography/LabTwo/bbs_binance_bits3.txt>

C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	P-VALUE	PROPORTION	STATISTICAL TEST
6	12	5	7	2	5	5	6	3	4	0.129620	54/55	Frequency
5	9	7	6	3	5	5	1	8	6	0.334538	54/55	BlockFrequency
7	7	9	3	4	6	5	3	3	8	0.401199	54/55	CumulativeSums
5	8	4	7	6	3	6	7	5	4	0.834308	54/55	CumulativeSums
4	5	7	5	9	7	4	4	5	5	0.798139	54/55	Runs
6	5	9	3	6	1	5	6	9	5	0.275709	55/55	LongestRun
3	7	5	6	4	4	2	8	7	9	0.366918	53/55	Rank
9	9	3	4	4	5	2	6	3	10	0.080519	55/55	FFT
4	4	4	8	5	6	5	6	7	6	0.924076	54/55	NonOverlappingTemplate
6	9	3	7	5	3	6	6	7	3	0.554420	54/55	NonOverlappingTemplate
4	5	9	8	7	4	5	4	1	8	0.249284	55/55	NonOverlappingTemplate
13	6	1	4	6	6	6	5	3	5	0.037566	53/55	NonOverlappingTemplate
5	6	6	11	3	2	4	3	8	7	0.129620	54/55	NonOverlappingTemplate
6	2	8	6	5	6	7	8	3	4	0.554420	53/55	NonOverlappingTemplate
13	6	4	6	4	2	3	5	8	4	0.032923	54/55	NonOverlappingTemplate
8	5	4	4	1	11	4	8	5	5	0.102526	53/55	NonOverlappingTemplate
5	4	3	7	7	7	7	8	6	1	0.401199	55/55	NonOverlappingTemplate
7	6	6	9	2	4	2	9	5	5	0.249284	53/55	NonOverlappingTemplate
8	2	7	7	7	4	5	4	6	5	0.678686	55/55	NonOverlappingTemplate
5	5	8	4	4	4	2	7	8	8	0.474986	55/55	NonOverlappingTemplate
9	3	1	5	3	12	3	6	6	7	0.019188	54/55	NonOverlappingTemplate
9	5	6	6	3	1	5	9	6	5	0.275709	55/55	NonOverlappingTemplate
3	8	8	3	6	4	6	7	5	5	0.678686	54/55	NonOverlappingTemplate
7	2	5	5	8	8	6	5	5	4	0.678686	54/55	NonOverlappingTemplate
7	6	8	6	7	5	6	5	4	1	0.595549	53/55	NonOverlappingTemplate
7	5	7	3	4	2	6	8	4	9	0.366918	52/55	NonOverlappingTemplate
10	5	4	6	6	7	2	6	6	3	0.401199	55/55	NonOverlappingTemplate
7	9	3	5	6	6	5	7	4	3	0.637119	54/55	NonOverlappingTemplate
2	4	8	5	2	7	8	5	7	7	0.366918	55/55	NonOverlappingTemplate
8	6	6	6	4	2	7	6	4	6	0.759756	55/55	NonOverlappingTemplate
7	5	7	3	6	6	2	4	7	8	0.595549	54/55	NonOverlappingTemplate
3	2	6	5	11	5	4	5	10	4	0.080519	54/55	NonOverlappingTemplate
3	9	8	5	4	3	7	7	6	3	0.401199	55/55	NonOverlappingTemplate

Рис. 7

7	8	4	3	10	5	6	4	3	5	0.366918	55/55	NonOverlappingTemplate
9	4	4	7	3	6	8	5	6	3	0.514124	53/55	NonOverlappingTemplate
9	6	7	5	3	5	6	4	3	7	0.637119	55/55	NonOverlappingTemplate
7	1	6	5	7	7	7	3	5	7	0.514124	54/55	OverlappingTemplate
8	8	4	4	1	8	7	6	3	6	0.275709	54/55	Universal
6	8	9	5	4	3	3	6	7	4	0.514124	55/55	ApproximateEntropy
5	1	2	3	0	6	5	4	3	2	0.275709	30/31	RandomExcursions
3	3	6	1	2	4	2	8	1	1	0.060239	30/31	RandomExcursions
4	3	6	3	5	0	2	3	2	3	0.500934	30/31	RandomExcursions
3	6	2	3	4	4	2	3	1	3	0.772760	31/31	RandomExcursions
5	6	7	1	3	0	2	4	2	1	0.060239	30/31	RandomExcursions
5	6	1	3	1	1	5	2	2	5	0.232760	31/31	RandomExcursions
2	2	3	2	8	3	5	2	3	1	0.195163	30/31	RandomExcursions
7	2	2	2	3	3	4	1	3	4	0.500934	30/31	RandomExcursions
2	3	4	0	5	4	5	3	3	2	0.637119	31/31	RandomExcursionsVariant
3	4	0	3	3	5	6	2	3	2	0.500934	31/31	RandomExcursionsVariant
2	2	3	1	5	7	2	6	2	1	0.134686	31/31	RandomExcursionsVariant
2	0	4	8	2	3	4	4	3	1	0.110952	31/31	RandomExcursionsVariant
2	3	3	3	4	5	2	4	1	4	0.888137	30/31	RandomExcursionsVariant
3	2	2	7	2	4	2	1	4	4	0.437274	30/31	RandomExcursionsVariant
1	3	3	4	3	5	1	6	4	1	0.437274	30/31	RandomExcursionsVariant
1	3	5	3	0	3	5	2	6	3	0.324180	30/31	RandomExcursionsVariant
3	4	1	4	1	4	7	2	4	1	0.275709	30/31	RandomExcursionsVariant
3	2	6	1	2	4	3	3	2	5	0.637119	31/31	RandomExcursionsVariant
4	3	2	3	2	2	0	3	3	9	0.060239	31/31	RandomExcursionsVariant
4	6	1	1	2	5	0	6	2	4	0.110952	31/31	RandomExcursionsVariant
4	5	2	2	6	3	4	4	1	0	0.324180	31/31	RandomExcursionsVariant
3	2	4	5	6	4	4	1	2	0	0.324180	31/31	RandomExcursionsVariant
4	2	4	4	7	3	2	1	3	1	0.378138	31/31	RandomExcursionsVariant
3	6	4	2	7	1	3	2	1	2	0.195163	30/31	RandomExcursionsVariant
5	3	4	6	2	4	2	3	1	1	0.500934	31/31	RandomExcursionsVariant
5	3	5	5	4	2	1	2	2	2	0.637119	31/31	RandomExcursionsVariant
8	6	6	8	5	2	3	4	7	6	0.554420	55/55	Serial
4	7	9	5	4	5	9	2	6	4	0.366918	54/55	Serial
8	7	5	6	5	9	8	2	2	3	0.202268	54/55	LinearComplexity

Рис. 8

Як бачимо, усі Nist тести також проходять.

Ось короткі та чіткі відповіді на контрольні питання для звіту:

1. Що таке PRNG та TRNG?

PRNG (Генератор псевдовипадкових чисел): Це алгоритм, який використовує математичну формулу для створення послідовності чисел. Вона виглядає як випадкова, але є повністю детермінованою: якщо знати початкове число (seed), можна точно відтворити всю послідовність.

TRNG (Генератор справжніх випадкових чисел): Базується на непередбачуваних фізичних процесах (тепловий шум, радіоактивний розпад, фотоелектричні явища). Він не має формули, тому його вихід неможливо передбачити або повторити.

2. Принцип роботи генератора BBS (Blum Blum Shub)

BBS — це криптографічно стійкий генератор, безпека якого базується на складності факторизації великих чисел.

Обираються два великих простих числа p та q (обидва $\equiv 3 \pmod 4$).

Обчислюється модуль $n = p \cdot q$.

Обирається випадкове число s (seed), взаємно просте з n.

Ітерація: $x_{i+1} = x_i^2 \pmod n$.

Вихід: на кожному кроці береться молодший біт (LSB) отриманого числа x.

3. Джерела ентропії для ініціалізації CSPRNG

Для "засолювання" (seeding) криптографічних генераторів використовують джерела з високим рівнем хаосу:

Часові інтервали (наносекунди між натисканнями клавіш або рухами миші).

Системні переривання та шум дискових операцій.

Дані від апаратних датчиків (температура процесора, шум мікрофона).

Зовнішні цифрові джерела (курси криптовалют, затримки мережевих пакетів).

4. Недоліки PRNG без достатньої ентропії

Передбачуваність: Якщо ентропії мало, зловмисник може вгадати seed шляхом перебору (Brute Force).

Відтворення: Мала кількість станів призводить до швидкого повторення послідовності (короткий період).

Вразливість ключів: Сгенеровані криптографічні ключі будуть слабкими, що дозволить зламати шифр без прямого атакування алгоритму шифрування.

5. Тести на перевірку випадковості

NIST STS: Стандартний набір із 15 тестів (Frequency, Runs, FFT, Linear Complexity тощо).

Покер Тест

Висновки

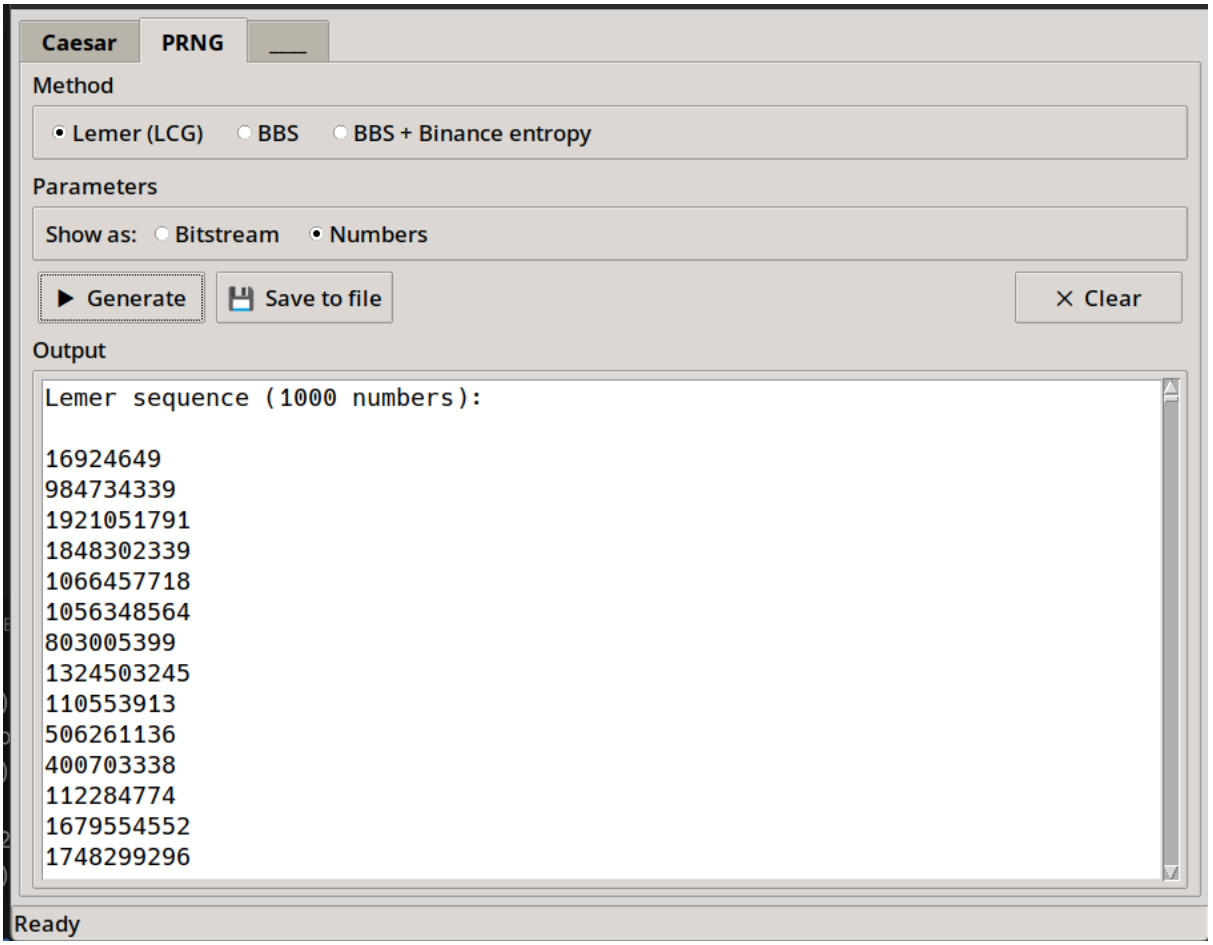
У ході роботи було реалізовано програмні моделі генератора Лемера та генератора BBS. Для кожного алгоритму сформовано бітові послідовності достатньої довжини та проведено їх перевірку за допомогою пакету статистичних тестів NIST з метою оцінки випадковості.

Додатково до генератора BBS було інтегровано власне джерело ентропії, яке формує початкове значення на основі зовнішніх даних та додаткового псевдовипадкового компонента. Отримані послідовності також були протестовані за методикою NIST.

На завершальному етапі виконано порівняльний аналіз результатів тестування класичного BBS та модифікованого варіанту з додатковою

ентропією, сформовано висновки щодо якості генерації та надано рекомендації щодо доцільності використання кожного підходу.

Також була зроблена графічна обгортка поверх написаних рішень:



Додаток А:

```
import math

import random

import requests

import time

import os





class NumericalGenerator:

    def __init__(self):

        pass



    def LemerGenerator(self, n, multi, modulus, seed):

        assert 0 < seed < modulus, "seed must be in range (0, modulus)"

        prev = seed

        generated_sequence = []

        for i in range(n):

            value = (multi * prev) % modulus

            generated_sequence.append(value)

            prev = value

        return generated_sequence



    def BBS(self, p, q, seed, number_of_bits):

        if p % 4 != 3 or q % 4 != 3 or p == q:

            print("Повинні виконуватись умови ...")
```

```
            return

        n = p*q



        if math.gcd(seed, n) != 1:

            print("Числа повинні бути взаємно простими (не мати спільних
дільників окрім 1) ...")

            return

        x = pow(seed, 2, n)

        bits = []

        for _ in range(number_of_bits):

            x = pow(x, 2, n)

            bits.append(str(x & 1))

        return "".join(bits)



    @staticmethod

    def get_entropy_from_binance(lemer_value):

        url =
"https://api.binance.com/api/v3/ticker/price?symbol=XMRUSDT"

        response = requests.get(url)

        price = response.json()['price']



        price_int = int(price.replace('.', ''))

        timestamp = time.time_ns()

        entropy = price_int ^ timestamp ^ lemer_value

        return entropy
```

```
    @staticmethod

    def numbers_to_bitsream(numbers, bit_width=31):

        bits = []

        for number in numbers:

            bits.append(format(number, f'0{bit_width}b'))

        return ''.join(bits)



    @staticmethod

    def bitsream_to_numbers(bitstream, chunk_size):

        numbers = []

        for _ in range(0, len(bitstream), chunk_size):

            binary_string = bitstream[_:_+chunk_size]

            number = int(binary_string, 2)

            numbers.append(number)

        return numbers





if __name__ == "__main__":

    num_gen = NumericalGenerator()



    # ── Lemer ──

    modulus = 2147483647

    multi = 16807

    n = 33000

    num_streams_lemer = 1
```

```
    seeds_lemer = [i * 1000 + 7 for i in range(1, num_streams_lemer +
1)]



    with
open("/home/siercig/University/Cryptography/LabTwo/lemer_bits3.txt",
'w') as file:

        for seed in seeds_lemer:

            generated = num_gen.LemerGenerator(n=n, multi=multi,
modulus=modulus, seed=seed)

            bits = num_gen.numbers_to_bitsream(generated)

            file.write(bits + '\n')



    # ── BBS ──

    p = 2147483659

    q = 2147483743



    num_streams_bbs = 55

    bits_per_stream = 1_000_000



    random.seed(42)

    seeds_bbs = random.sample(range(2, p * q), num_streams_bbs)



    with
open("/home/siercig/University/Cryptography/LabTwo/bbs_bits3.txt", 'w')
as file:

        for s in seeds_bbs:
```

```
            if math.gcd(s, p * q) == 1:

                bits = num_gen.BBS(p=p, q=q, seed=s,
number_of_bits=bits_per_stream)

                if bits:

                    file.write(bits + '\n')



    print("Обычный BBS готово")



    # ── BBS с энтропией Binance + Lemer ──

    lemer_entropy_sequence = num_gen.LemerGenerator(n=num_streams_bbs,
multi=16807, modulus=2147483647, seed=99991)



    with
open("/home/siercig/University/Cryptography/LabTwo/bbs_binance_bits3.tx
t", 'w') as file:

        for i in range(num_streams_bbs):

            raw_entropy =
num_gen.get_entropy_from_binance(lemer_entropy_sequence[i])



            s = (raw_entropy % (p * q - 2)) + 2



            while math.gcd(s, p * q) != 1:

                s = (s + 1) % (p * q)



            bits = num_gen.BBS(p=p, q=q, seed=s,
number_of_bits=bits_per_stream)

            if bits:
```

```
                file.write(bits + '\n')
            print(f"BBS Binance Stream {i + 1}/{num_streams_bbs} |
seed={s}")
            time.sleep(0.1)
    print("BBS Binance готово")
```